YUZEE AI TOKEN LAB · DOCUMENTATION

Multi-Agent mode

What was added to GitHub, how the new mode works, and the pixel-art team window · 5 October 2026

In one paragraph. Multi-Agent is a third way to run Oala, next to Vanilla (AI Studio) and EXP-RAG. When a question needs it, a small team of specialist AI helpers works on it at the same time. Each one has one narrow job and looks up its own evidence in the Yuzee data warehouse. Their findings are checked and merged, and Oala then writes one answer with clear next steps. Simple questions skip the team. The user can watch the team work in the chat and in an optional pixel-art office window.

8 specialists
Profile, Jobs, Skills, Market, Company, Learning, Pathway, Opportunity

Works in parallel
Specialists run side by side, about 2 s after the data lookup

Evidence-checked
Anything a specialist cannot back with a record is removed

1. What we updated on GitHub

Repository	`ashik-yuzee/Yuzee-AI-Lab`
Branch	`feature/exp-rag-intelligence-streaming` (not merged into `main` yet)
Commit	`c37da0d` · "Add Multi-Agent mode with parallel specialists and pixel team window"
Size	81 files, +51,092 / −58 lines. About 45,000 of the added lines are saved evaluation data (answers and judge scores); the code itself is about 2,800 lines.

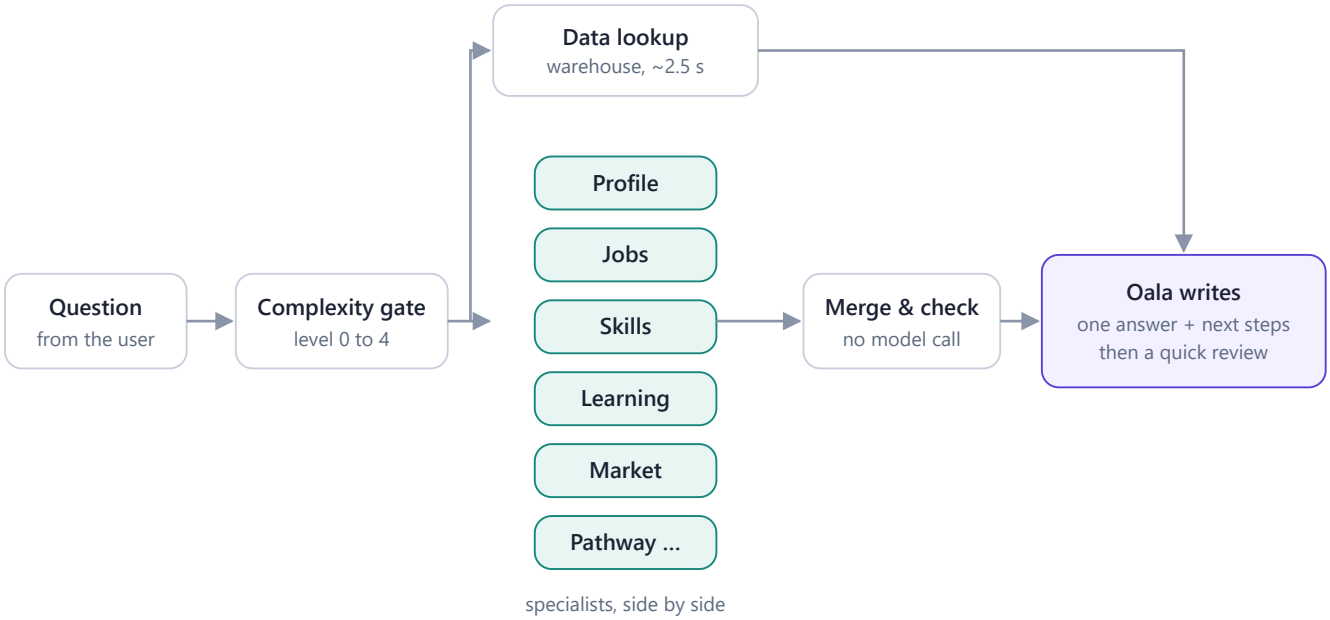
What is in the commit

Area	Files	What it does
Multi-Agent engine new	`src/agents/router.ts` `src/agents/registry.ts` `src/agents/specialists.ts` `src/agents/orchestrator.ts` `src/agents/merger.ts` `src/agents/contract.ts`	Decides how complex a question is, defines the 8 specialists, runs them in parallel with timeouts, checks their evidence, merges results and prepares the package Oala writes from. `contract.ts` holds the shared types and live events.
Server	`server.ts` , `src/rag/pipeline.ts` , `src/services/db.ts`	Adds the `MULTI_AGENT` mode on top of the EXP-RAG pipeline, streams team progress live, saves the team's work on each message (local store and Postgres), bills each specialist call with the turn, and adds two endpoints.

Chat interface	`src/components/agents/AgentActivityPanel.tsx`, `agents.css`, `src/agents/viewModel.ts`, `Navbar.tsx`, `Composer.tsx`, `ChatArea.tsx`, `TokenLabContext.tsx`, `api.ts`, `types.ts`	"Multi-Agent" in the mode menu, the live team panel above each answer, expandable evidence, and next-step buttons. The message box puts the text on its own row on phones.
Pixel team window new	`AgentTeamButton.tsx`, `PixelAgentsWindow.tsx`, `pixel/office.ts`, `pixel/LICENSE`, `public/pixel-agents/*.png` (20 sprites, ~49 KB)	The button beside the message box and the draggable office window (section 3).
Shared fix	`src/protocol/presentationDefaults.ts`	Repairs a list or table block that is missing its optional intro text, which made one answer fail. Helps every mode.
Tests new	`tests/multi-agent.test.ts`, `agents-view-model.test.ts`, `pixel-agents.test.ts`	38 routing cases, parallel timing, timeouts, cancellation, evidence checks, merging, next actions, UI states, window docking and office layout. No live AI calls.
Evaluation and reports	`scripts/collect-modes.ts`, `scripts/multi-agent-report.ts`, `docs/reports/data/multi-agent/`, `docs/reports/multi-agent-comparison.pdf`	Collects answers from all modes on an isolated server, stores blind judge scores, and builds the comparison PDF.
Docs	`docs/MULTI-AGENT.md`, `package.json` scripts	Technical reference. New scripts: `test:multi-agent`, `test:agents-ui`, `test:pixel-agents`, `report:multi-agent`.

2. How the Multi-Agent mode works

Multi-Agent is built on EXP-RAG. Oala still uses EXP-RAG's short cached core prompt, picks only the guidance and warehouse evidence this question needs, keeps a rolling memory of the chat, and checks the answer before it is shown. The new part is the team that works before Oala writes.



Simple questions (level 0) go straight to Oala. Quick lookups (level 1) use one data check with no extra AI call. Personal or multi-part questions (level 2–4) bring in the team.

Step 1 — The complexity gate decides how much help is needed

Level	Example	What happens
0 · direct	"What does RPL mean?"	Oala answers straight away. No team, no extra cost.
1 · lookup	"How many UX designer jobs are in Perth?"	One specialist does the data lookup only (no AI call). A one-line status is shown.
2 · personal	"Am I suitable for UX designer jobs?"	Profile plus one or two domain specialists.
3 · planning	"I want to move from hospitality into UX design."	Five or six specialists, including Pathway, run side by side.
4 · action	"Find jobs I can realistically apply for and help me close the gaps."	The full team, plus opportunities beyond the request.

The gate uses fast rules, so it adds no delay. When the warehouse's own planner (which runs anyway) understands the question better, it can add specialists, never remove them.

Step 2 — Specialists work at the same time

Specialist	Its one job	Waits for
Profile	Reads what the user has told us about themselves and lists what is missing. The only helper that sees personal profile data.	nothing

Jobs	Job records, common titles, recurring requirements, and related roles the user did not search for.	data lookup
Skills	Compares the user with what employers ask for, and ranks the gaps.	Profile
Market	Demand signals, each labelled historical, current, inferred or uncertain. Never invents a trend.	data lookup
Company	What a named employer asks for. Says plainly when there is no data about that employer.	data lookup
Learning	The smallest course, skill set or unit that closes a gap, with local providers when a place is named.	data lookup
Opportunity	Related roles, traineeships and placements that could fit better than the literal request.	Profile
Pathway	Up to three realistic routes, the best one, and the best next action.	Profile

Each specialist is one fast AI call (Gemini 3.5 Flash-Lite) with a narrow task and a small, fixed answer format, so each takes about 2 seconds. Each one also runs its own search of the warehouse catalogue (the EXP-RAG index of qualifications, courses, units, occupations and providers), so it fetches the data it needs for its own job.

Step 3 — Evidence is checked and merged

- Every finding must cite records the specialist was actually given. Findings that cite nothing real are removed, and that removal is shown as a warning.
- Facts (from records) are kept apart from interpretations (the specialist's judgement).
- Duplicates are removed without inflating confidence. If two specialists disagree, both views are kept as nuance.
- Old records are flagged, and missing information is listed.
- A short answer plan is prepared: best route, best next step, main gaps, and options beyond the request.

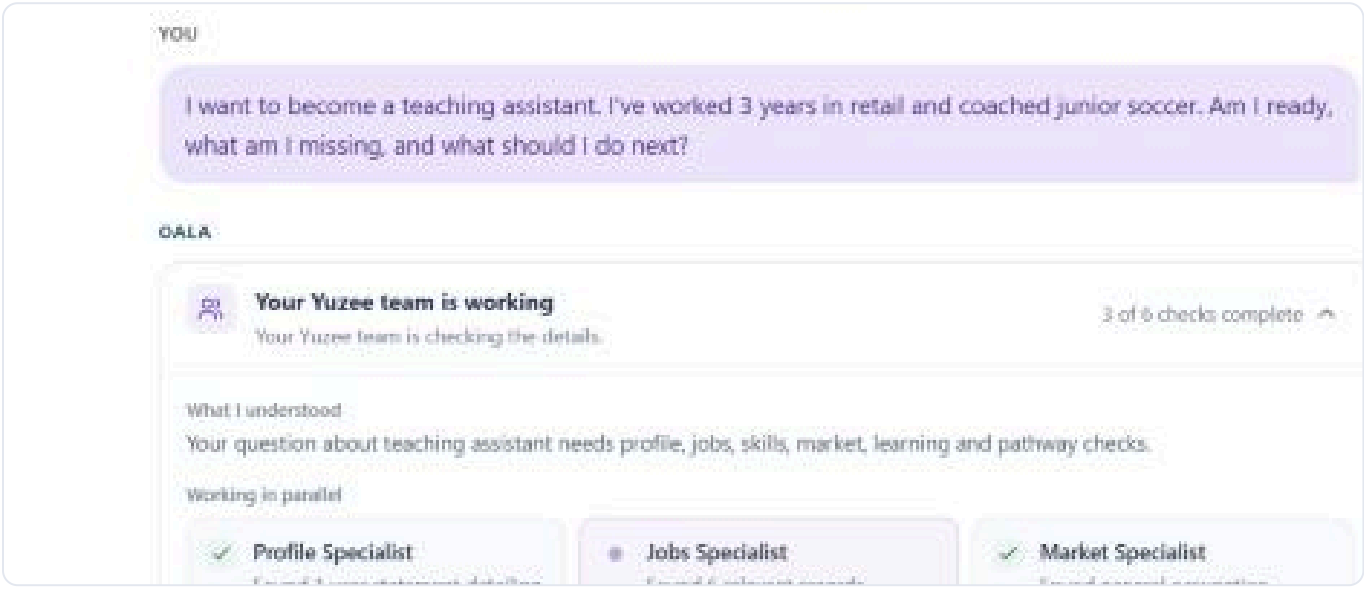
Step 4 — Oala writes one answer

Oala receives the merged findings next to the usual EXP-RAG evidence and writes **one** answer, never a list of agent reports. The answer gives the direct answer first and says in one sentence what was checked. It ties advice to what the user said, names its sources, and gives the best next step. It asks at most one useful question. If a specialist did not finish, Oala says so plainly instead of guessing. A short review then checks the answer before it is shown.

Step 5 — Failures, cancelling and saving

- **Timeouts:** each specialist has a limit (5–6.5 s). A slow or failed specialist never breaks the answer; it is marked "didn't finish".
- **Cancelling:** pressing Stop, or sending a new question in the same chat, stops the specialists still running.
- **Saving:** the team's work (plan, findings, sources, timings, cost) is saved on the message. It reappears after a reload, and a profile review can be reused later in the same chat.
- **Cost tracking:** every specialist call is billed with the turn and shown in the token telemetry.

What the user sees



While the team works, the panel lists the specialists with their status. Ones that run at the same time appear side by side. When the answer is ready, the panel collapses to "Oala has brought everything together" and next-step buttons appear (only ones that lead somewhere that works).

- **Expandable results:** each specialist card opens to show its facts, interpretations, sources and how recent they are, its confidence, and anything missing.
- **Next steps:** buttons such as "Compare UX courses", "Build this pathway" or "Add your missing details" send a follow-up, open the mini pathway, or open the profile editor.
- **Accessibility and phones:** status changes are announced to screen readers, buttons work with the keyboard, animation is off for reduced motion, and the panel becomes a simple list on phones.

Settings and switches

Setting	Effect
Mode menu → **Multi-Agent**	Turns the mode on for a new conversation. Like other modes, it is fixed once the chat has started.
Thinking menu → **Medium** (default)	Best answer quality.
Thinking menu → **Low**	The fastest option of any mode (about 13.7 s), with lower answer quality in blind tests.
`MULTI_AGENT_DISABLED` (server)	Switches specialists off, for example `COMPANY_SPECIALIST,MARKET_SPECIALIST`.
`GET /api/agents/registry`	Lists every specialist: version, abilities, data it may read, time limit and permissions. No specialist can write, apply or send anything.
`POST /api/conversations/:id/agent-events`	Records which results people open and which next steps they click (for measuring usefulness).

How it compares (blind test, 11 questions)

Mode	Quality (average place, 1 = best)	Typical time	First words	Cost per answer
Vanilla (AI Studio)	1.82	17.6 s	10.6 s	$0.0039

Multi-Agent	1.91	16.6 s	9.5 s	$0.0033
EXP-RAG	1.95	14.8 s	8.7 s	$0.0027

On quality the three modes are effectively tied. Multi-Agent is faster than Vanilla and slower than EXP-RAG: the specialists work in parallel, but the final answer is still written by one model after they finish, and that careful writing is what keeps quality high. The full comparison is in `docs/reports/multi-agent-comparison.pdf`.

Running and testing

Command	What it checks
`npm run test:multi-agent`	Routing, parallel timing, timeouts, cancellation, evidence checks, merging, next actions
`npm run test:agents-ui`	Team panel states and markup
`npm run test:pixel-agents`	Window docking and office layout
`npm run report:multi-agent`	Rebuilds the comparison PDF from saved evaluation data

3. Additional: the pixel-art team window

An optional, playful view of the same real work. It shows the team as characters in a small pixel-art office, built with the open-source **Pixel Agents** project.



Before a question: every team member has a labelled desk and Oala is at hers.



During a question: Profile and Pathway are done (✓), Jobs and Skills are typing, Market waits for its turn (…), Learning did not finish (!), and Company and Opportunity were not needed.

How to use it

- **Only in Multi-Agent mode:** a team button appears beside the message box, with a badge counting the specialists working right now.
- **Open and close:** click the button to open the window; press Esc or the × to close it.
- **Move it:** drag the window by its header. When you let go it snaps to the nearest edge or corner and stays there, even after a reload. Keyboard users can move it with the arrow keys on the grip.
- **Minimise:** the arrow button collapses the window to its header. On phones the window is a bottom sheet.

What the office shows

In the office	Meaning
Labelled desk for every member	Oala in the middle of the front row, then Profile, Jobs, Skills and Market, with Company, Learning, Pathway and Opportunity behind. Seats never move.
Empty desk, dimmed nameplate	That specialist was not needed for this question.
Standing by the desk, "…" bubble	Waiting for its turn (for example Skills waits for Profile).
Sitting and typing, screen on	Working now. The bubble shows its real status, such as "Searching the course catalogue".
✓ bubble, then seated	Finished.
"!" bubble	Did not finish; nothing from it was used.
Oala typing	Understanding the question, or bringing the results together and writing.

Nothing in the office is staged: every pose comes from the same live events as the chat panel. A text list under the office repeats the same information for screen readers. Animation stops when nothing is changing, when the window is closed, or when the tab is hidden.

Artwork and licence



- **Renderer and furniture:** adapted from **Pixel Agents** (github.com/pixel-agents-hq/pixel-agents), MIT licence, © 2026 Pablo De Lucca. The licence is in `src/components/agents/pixel/LICENSE`.
- **Characters:** based on JIK-A-4's Metro City pack (free to use). Three colour variants were made so all nine team members look different.
- **Credit:** both are credited in the window footer.

4. Known limits and next steps

Gemini credits. The project's Gemini prepaid credits ran out on 5 October 2026 (error 402). Until they are topped up in AI Studio, no mode can answer, and the EXP-RAG test that calls Gemini fails.

- **Speed versus quality:** lowering Oala's thinking makes Multi-Agent the fastest mode, but the answers rank lower, so the default keeps careful thinking. The first version (Pathway waiting for all specialists, more detailed notes) ranked best on quality but was about 5 s slower than EXP-RAG. It can be re-tested once credits are back.
- **Not merged yet:** the work is on the feature branch, not `main`.
- **Journey:** the spec's later phases (persistent agents that keep working between chats, a saved personal journey) are not built. Team results are saved on each message.
- **Small test:** quality was judged on 11 questions by two AI judges. A larger set would give firmer numbers.

Technical reference: `docs/MULTI-AGENT.md` · Comparison report: `docs/reports/multi-agent-comparison.pdf` · Specification: Yuzee Agentic Orchestration v1.0